

Atividade 10: Análise e Projeto de Sistemas

Introdução

Um projeto de sistemas é um conjunto de atividades organizadas para criar ou modificar um software. Ninguém constrói uma casa sem planta, e com software é a mesma coisa. O sistema é pensado, desenhado, discutido e só depois vira código. Projetar um sistema envolve entender o problema do cliente, definir requisitos, escolher tecnologias, modelar processos e documentar cada etapa.

Um software precisa ser planejado antes de ser desenvolvido por um motivo simples: evitar retrabalho. Se você sai codando sem saber o que precisa fazer, a chance de refazer tudo depois é enorme. O planejamento mostra o que o cliente realmente quer (muitas vezes ele mesmo não sabe direito), ajuda a estimar prazos e custos, identificar riscos e garantir que o produto final atenda o esperado. Sem planejamento, projetos viram um caos. Prazos estouram, orçamentos disparam, cliente fica insatisfeito.

A documentação em projetos de TI registra todas as decisões tomadas ao longo do caminho. Ela é a memória do projeto. Um desenvolvedor novo que entra na equipe consulta a doc pra entender o sistema. Quando surge um bug, a documentação mostra por que uma decisão foi tomada. Anos depois, quando o sistema precisa de atualização, a doc é o ponto de partida. Sem ela, o conhecimento fica na cabeça de algumas pessoas. Quando elas saem da empresa, o conhecimento vai junto.

1. Conceitos de Arquitetura de Sistemas

O que é arquitetura de software

Arquitetura de software é a estrutura fundamental de um sistema: como os componentes se organizam, se relacionam e quais princípios guiam seu design e evolução. É o esqueleto do software. Define as partes principais e como elas se comunicam, sem entrar nos detalhes de implementação de cada parte.

Qual sua importância

A arquitetura define atributos como desempenho, escalabilidade, segurança, facilidade de manutenção e portabilidade. Uma boa arquitetura permite adicionar funcionalidades novas, escalar quando a demanda cresce e corrigir problemas sem quebrar o sistema inteiro. Uma arquitetura ruim torna o sistema lento, difícil de manter e cheio de bugs. Por isso a escolha da arquitetura é uma das decisões mais importantes em um projeto de software.

Diferença entre arquitetura e implementação

Arquitetura é o "que" e o "como macro": quais são os módulos, como se comunicam, qual o fluxo de dados. Implementação é o "como micro": o código em si, as classes, funções, bibliotecas que você escolhe. A arquitetura define as regras gerais; a implementação segue essas regras. Decidir que o sistema terá uma camada de apresentação separada da camada de negócio é uma decisão arquitetural. Escrever o HTML e CSS da tela de login é implementação.

Três exemplos de arquiteturas atuais

1. Arquitetura Monolítica: o sistema inteiro é uma única aplicação. Interface, lógica e dados rodam no mesmo processo. É simples de desenvolver no início, mas fica difícil de escalar e manter conforme cresce.
2. Arquitetura de Microsserviços: o sistema é dividido em serviços pequenos e independentes, cada um com seu próprio banco de dados. Eles se comunicam por APIs (geralmente HTTP ou mensageria). Dá pra escalar cada serviço separadamente e usar tecnologias diferentes em cada um.
3. Arquitetura em Camadas (Layered): o sistema é organizado em camadas horizontais, tipo apresentação, negócio e dados. Cada camada se comunica só com a camada imediatamente abaixo. É a arquitetura mais tradicional, ainda muito usada em sistemas corporativos.

Exemplo prático

Um exemplo prático é o sistema de uma Biblioteca Online. Cada área pode ser um serviço independente: catálogo de livros, cadastro de usuários, controle de empréstimos, reservas e notificações. Cada serviço tem seu banco de dados e pode ser atualizado separadamente sem afetar os outros.

2. Modelagem de Contexto

O que é

Modelagem de contexto é uma técnica que coloca o sistema como uma "caixa preta" no centro de um diagrama e mostra seus relacionamentos com entidades externas, como pessoas, outros sistemas e organizações. O foco é no que está fora do sistema, não dentro.

Objetivo

O objetivo é definir o escopo do sistema: onde ele começa e termina, quem interage com ele e quais informações entram e saem. Ajuda a garantir que todos no projeto tenham a mesma visão do que o sistema vai fazer (e do que não vai).

Informações representadas

O diagrama de contexto mostra o sistema central como um único retângulo, as entidades externas representadas por figuras, e os fluxos de dados entre o sistema e cada entidade, com setas indicando o que trafega.

Quando é utilizada

A modelagem de contexto é usada nas fases iniciais do projeto, durante o levantamento de requisitos. Geralmente é o primeiro diagrama criado e serve de ponto de partida para modelagens mais detalhadas.

Diagrama de Contexto: Biblioteca Online



3. Modelagem de Comportamento

O que significa modelar o comportamento de um sistema

Modelar o comportamento é representar como o sistema reage a eventos e interações ao longo do tempo. A modelagem estrutural mostra "o que" compõe o sistema. A modelagem comportamental mostra "como" ele funciona: quais funcionalidades oferece, em que ordem as coisas acontecem, como responde ao usuário.

Diagramas normalmente utilizados

Os diagramas de comportamento na UML são: Diagrama de Casos de Uso (funcionalidades na visão do usuário), Diagrama de Atividades (fluxo de processos), Diagrama de Sequência (interação entre objetos ao longo do tempo) e Diagrama de Estado (ciclos de vida de um objeto).

Para que servem os Casos de Uso

Casos de Uso descrevem as funcionalidades do sistema do ponto de vista do usuário. Eles respondem "o que o sistema faz?" sem se preocupar com "como faz". Cada caso de uso descreve uma interação completa entre um ator (usuário ou outro sistema) e o sistema para atingir um objetivo específico.

O que é um Diagrama de Atividades

O Diagrama de Atividades mostra o fluxo de atividades de um processo: a sequência de passos, decisões (sim/não), ações paralelas e o início/fim do fluxo. É parecido com um fluxograma, mas segue a notação UML.

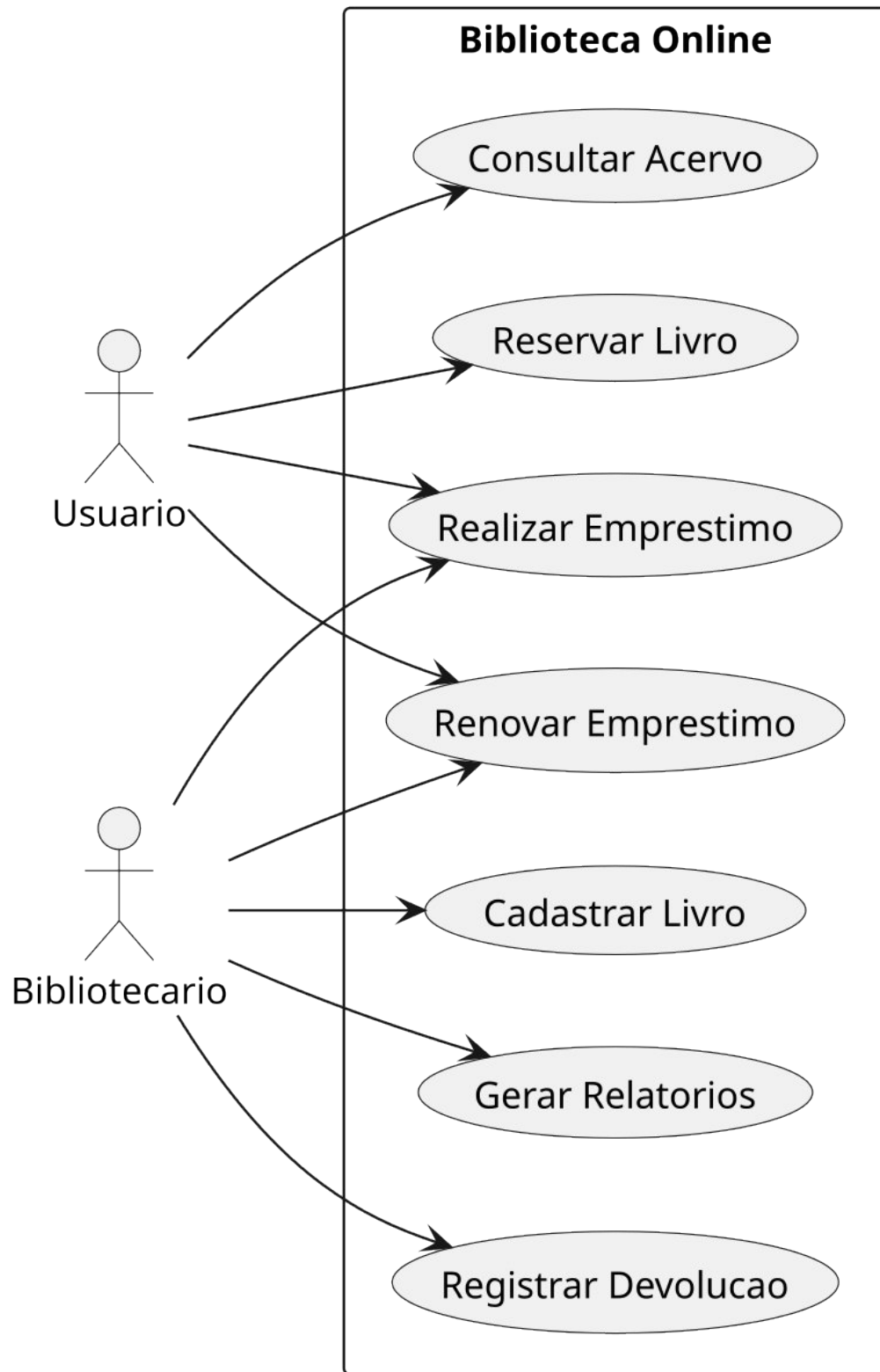
Exemplo de Caso de Uso: Realizar Empréstimo

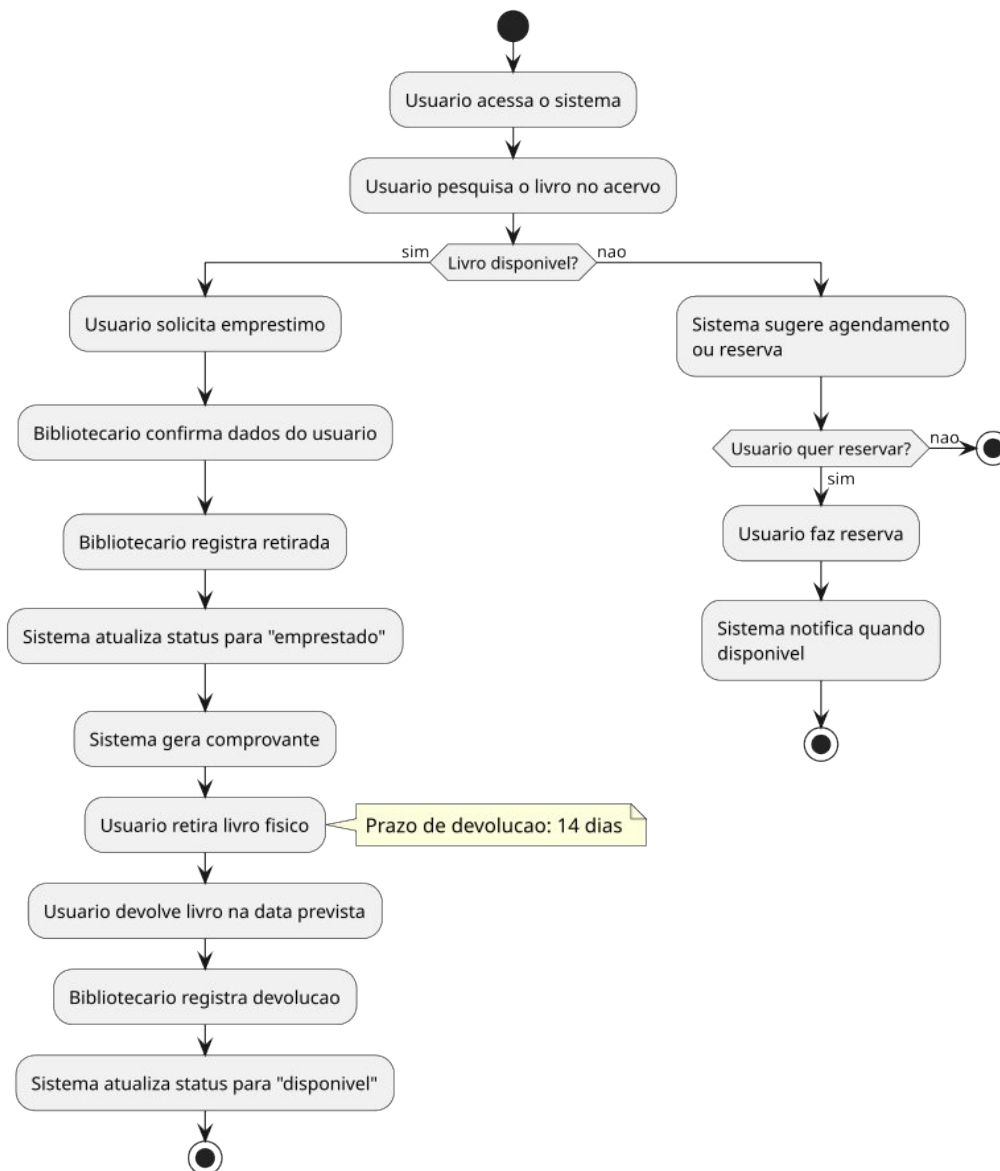
Ator: Usuário

Objetivo: Pegar um livro emprestado na biblioteca

Fluxo principal:

1. Usuário acessa o sistema com seu login
2. Usuário consulta o acervo por título, autor ou categoria
3. Usuário seleciona o livro desejado
4. Sistema verifica a disponibilidade do livro
5. Usuário solicita o empréstimo
6. Sistema registra o empréstimo com data de retirada e devolução
7. Sistema atualiza o status do livro para "emprestado"
8. Sistema envia comprovante do empréstimo por email





4. Modelagem de Processos

O que é modelagem de processos

Modelagem de processos é a prática de representar visualmente as etapas de um processo de negócio. Mostra como o trabalho flui dentro de uma organização, quem faz o quê, em que ordem e quais decisões são tomadas no caminho.

Diferença entre processo e atividade

Um processo é um conjunto de atividades coordenadas que transformam entradas em saídas e geram valor para o cliente. Uma atividade é uma etapa específica dentro desse processo.

"Realizar Empréstimo" é um processo. "Consultar o acervo" e "Registrar a retirada" são atividades dentro desse processo.

O que é BPM

BPM (Business Process Management) é uma disciplina de gestão que trata processos de negócio como ativos estratégicos. Envolve modelar, executar, monitorar e otimizar processos continuamente para melhorar a eficiência e os resultados da organização.

O que é BPMN

BPMN (Business Process Model and Notation) é uma notação padronizada (criada pela OMG) para modelar processos de negócio. Define símbolos específicos para eventos (início, fim, intermediários), atividades (tarefas, subprocessos), gateways (decisões, paralelismo) e fluxos de sequência. É o padrão mais usado atualmente.

Exemplo: Processo "Realizar Empréstimo"

Etapas em ordem lógica:

1. Usuário acessa o sistema da biblioteca
2. Usuário pesquisa o livro desejado no acervo
3. Sistema exibe a disponibilidade e localização do livro
4. Usuário vai até a estante e retira o livro físico
5. Usuário leva o livro ao balcão da biblioteca
6. Bibliotecário confirma os dados do usuário no sistema
7. Bibliotecário registra a retirada com data de devolução
8. Sistema atualiza o status do livro para "emprestado"
9. Sistema gera um comprovante para o usuário
10. Na data prevista, usuário devolve o livro
11. Bibliotecário registra a devolução no sistema
12. Sistema atualiza o status do livro para "disponível"
13. Se atrasar, sistema calcula multa proporcional

5. A Importância da Documentação

Por que documentar um projeto

Documentar um projeto registra decisões, requisitos, regras de negócio, arquitetura e fluxos do sistema. A documentação é a memória oficial do projeto. Permite que qualquer pessoa da equipe entenda o sistema sem depender de quem o construiu.

Problemas sem documentação

Sem documentação, o conhecimento fica centralizado em poucas pessoas. Quando elas saem, o conhecimento vai embora. A manutenção vira um jogo de adivinhação. Desenvolvedores novos

demoram muito mais para produzir. Decisões antigas são repetidas ou contraditas por falta de registro. Bugs aparecem porque ninguém sabe como o sistema deveria se comportar.

Quem usa a documentação

Analistas de sistema usam para validar requisitos. Desenvolvedores usam para entender o que implementar. Testers usam para criar casos de teste. Arquitetos usam para garantir que a implementação segue o projeto. Gerentes usam para acompanhar o andamento. Suporte usa para atender o usuário final.

Opinião pessoal

Na minha opinião, documentação é aquela coisa que todo mundo sabe que deveria fazer, mas ninguém tem tempo. Só correm atrás dela quando precisam e não tem. Já vi equipe perder dias tentando entender uma decisão antiga porque não havia registro nenhum. Não precisa ser um documento enorme e burocrático. Um README bem escrito, uns diagramas atualizados e comentários nos pontos críticos já resolvem. O negócio é documentar o suficiente pra qualquer pessoa da equipe conseguir dar manutenção sem pânico.

6. Aplicação na Profissão de TI

O analista de sistemas usa arquitetura, modelagem, documentação e processos em todas as fases do desenvolvimento.

Arquitetura: o analista ajuda a definir a arquitetura do sistema com o arquiteto, escolhendo a abordagem que faz sentido para o problema (monolítica, microsserviços, camadas). Ele precisa entender as consequências de cada escolha para explicar ao cliente e à equipe.

Modelagem: o analista cria e interpreta diagramas (contexto, casos de uso, atividades, sequência) para traduzir as necessidades do cliente em algo que a equipe técnica entenda e implemente. É a ponte entre o negócio e a tecnologia.

Documentação: o analista registra requisitos, regras de negócio, fluxos e decisões de projeto. Essa documentação vira contrato entre equipe e cliente, e guia para desenvolvimento e testes.

Processos: o analista mapeia e modela os processos de negócio do cliente pra entender como o sistema deve apoiá-los. Ele identifica gargalos, retrabalhos e oportunidades de automação, propondo melhorias nos processos antes mesmo de pensar no software.

No fim das contas, o analista de sistemas traduz problemas de negócio em soluções de software usando essas quatro ferramentas como base do trabalho.

Conclusão

Com essa pesquisa, aprendi que desenvolver um sistema vai muito além de sentar e programar. Um software bem feito começa antes do código. Começa com planejamento, modelagem, conversas com o cliente, desenhos e diagramas que ajudam a entender o problema antes de tentar resolvê-lo.

O assunto que mais me chamou atenção foi modelagem de processos com BPMN. Não imaginava que existia uma notação tão específica pra representar processos de negócio. Ver como cada etapa, decisão e fluxo pode ser representado graficamente mudou a forma como enxergo processos do dia a dia.

Outro ponto interessante foi a diferença entre arquitetura e implementação. Antes eu achava que arquitetura era só um nome bonito pra estrutura do código. Agora entendo que arquitetura é uma camada de decisão muito mais estratégica, que impacta o sucesso ou fracasso de um sistema.

Esse conhecimento vai me ajudar na área de TI, principalmente se eu seguir como analista ou desenvolvedora. Saber planejar antes de executar, entender a importância de registrar decisões e modelar processos e comportamentos são habilidades que fazem diferença entre um profissional que só programa e um que entrega soluções completas. Mesmo que eu nunca precise desenhar um diagrama de atividades na mão, o raciocínio de pensar no fluxo antes de codar já vale o aprendizado.